

Objectives

- Upon completing this module, you will be able to:
 - Describe ADO.NET
 - Define data providers
 - Define a connection object
 - Define a command object
 - Define a DataReader object
 - Define DataSet and DataAdapter objects
 - Describe the use of SqlHelper in Data Access Application Block
 - Describe the exception hierarchy in ADO.NET
 - Introduction to Transactions in ADO.NET



Agenda

- Understanding ADO.NET
- The SqlConnection Object
- The SqlCommand Object
- Reading Data with the SqlDataReader
- Implementation of Connected Data Management
- Working with Disconnected Data
- DataTable Object
- DataColumn Object
- DataRow Object
- Using Stored Procedures
- Implementation Using Stored Procedure
- Exception Handling in ADO.NET



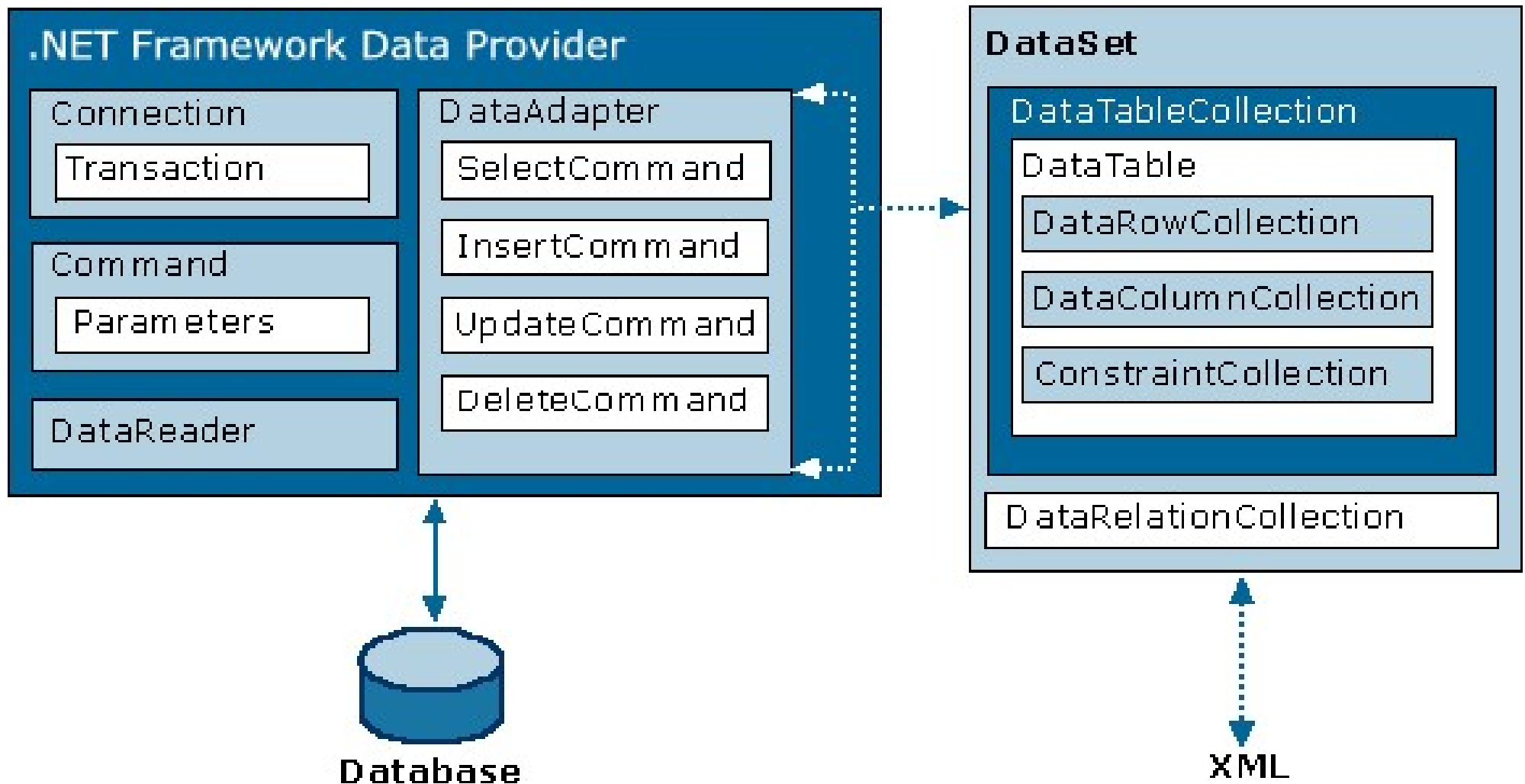
Understanding ADO.NET

- ADO.NET provides **consistent access** to data sources such as Microsoft SQL Server as well as to other **data sources**.
- ADO.NET is used to connect to data sources and retrieve, manipulate, and update the data that they contain.
- ADO.NET includes **.NET Framework data providers** for connecting to a database, executing commands, and retrieving results.
- The ADO.NET classes are found in **System.Data.dll**, and are integrated with the XML classes found in System.Xml.dll.
- ADO.NET provides functionality to developers writing managed code similar to the functionality provided to native COM developers by ADO (ActiveX Data Objects).

Features of ADO.Net

- Object oriented data access (Classes to connect, query, and administer data sources)
- Unique/Uniform data access (Uniform API for accessing data from different sources)
- Supports both command-based and disconnected data (Enables data to be accessed through a connection or a data set)
- Data binding (Binds a data source to ASP.NET, Windows Forms, or WPF controls)

Understanding ADO.NET Architecture

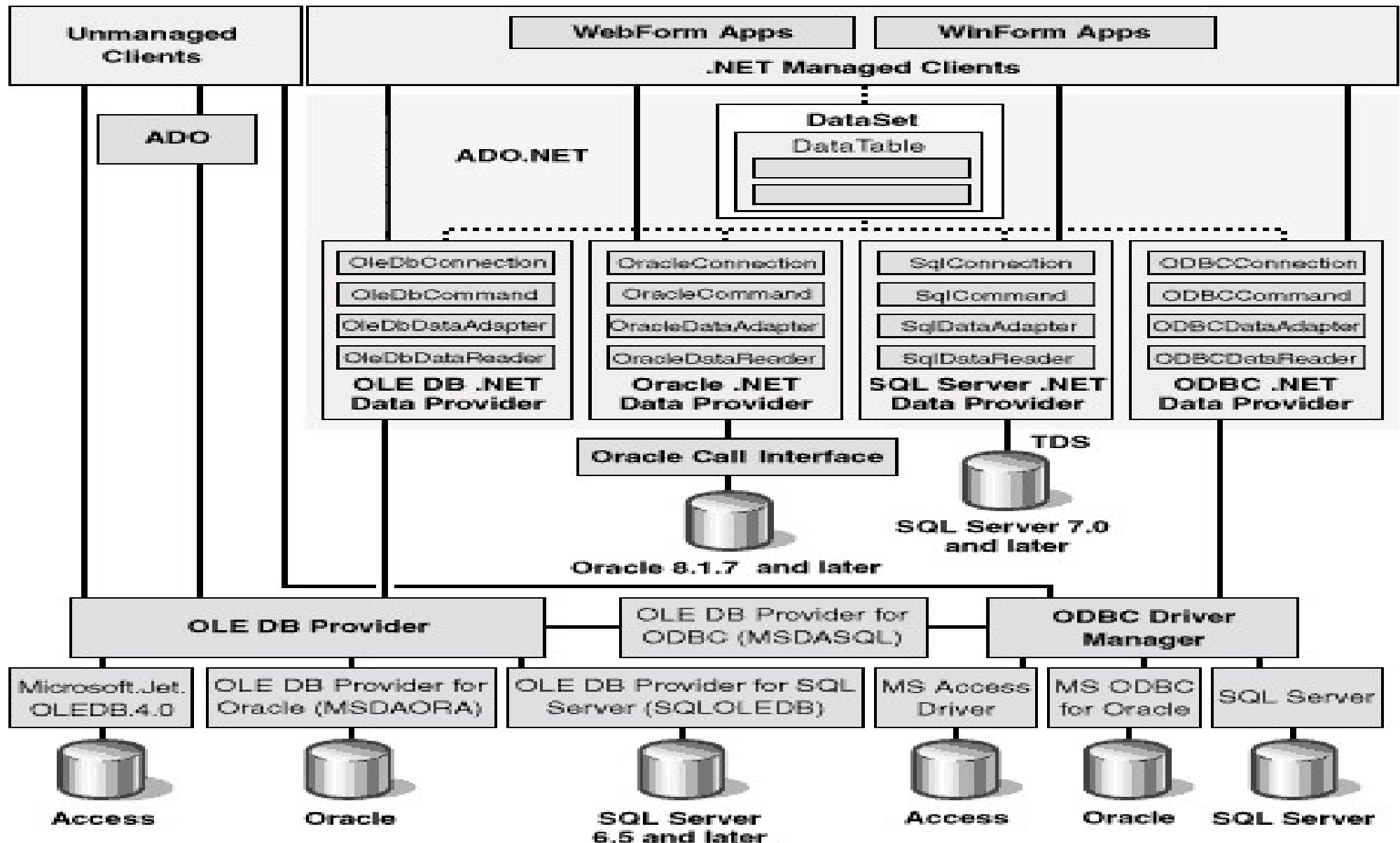


Understanding ADO.NET Data Providers

- The .NET Framework data provider is used for connecting to a database, executing commands, and retrieving results.
- The following table lists the data providers that are included in the .NET Framework.

Provider Name	API Prefix	Data Source Description
ODBC Data Provider	Odbc	Data Sources with an ODBC interface. Normally older databases.
OleDb Data Provider	OleDb	Data Sources that expose an OleDb interface, i.e. Access, or Excel.
Oracle Data Provider	Oracle	For Oracle databases.
SQL Data Provider	Sql	For interacting with Microsoft SQL Servers.
Entity Client Provider	Entity	Provides access for Entity Data Model (EDM) applications.

Understanding ADO.NET Data Providers – Data Access Stack



Understanding ADO.NET Objects

- ADO.NET includes many objects that you can use to work with data. The primary objects are:
 - The SqlConnection Object – Used for connecting to the database.
 - The SqlCommand Object – Used for specifying the actions to be performed on the database.
 - The SqlDataReader Object – Used to obtain the results in a fast-forward only stream.
 - The DataSet Object – Used for in-memory representation of the data.
 - The SqlDataAdapter Object – Used to manage data in a disconnected mode.

SqlConnection Object

- SqlConnection is an object used to connect to a SQL Server database.
- ADO.NET **Connection Strings** contain certain **key/value pairs** for specifying how to make a database connection.
- They include the location and name of the database, and security credentials.

Parameter Name	Description
Data Source	Identifies the server. Could be local machine, machine domain name, or IP Address.
Initial Catalog	Database name.
Integrated Security	Set to SSPI to make connection with user's Windows login.
User ID	Name of user configured in SQL Server.
Password	Password matching SQL Server User ID.

SqlConnection Object (cont.)

- Connection Strings Format:
 - `SqlConnection conn = new SqlConnection("Data Source=DatabaseServer;Initial Catalog=Northwind;User ID=YourUserID;Password=YourPassword");`
 - `SqlConnection conn = new SqlConnection("Data Source=(local);Initial Catalog=Northwind;Integrated Security=SSPI");`
- The sequence of operations that occur during the lifetime of a SqlConnection are:
 1. Instantiate the SqlConnection.
 2. Open the connection.
 3. Pass the connection to other ADO.NET objects.
 4. Perform database operations with the other ADO.NET objects.
 5. Close the connection.

SqlCommand Object

- A SqlCommand object allows you to specify what **type of interaction** you want to perform with a database.
- The SqlCommand can be used for both connected and disconnected management of data.
- There are several ways to create an SqlCommand object including:
 - Initialize a new instance of the SqlCommand class.
`SqlCommand myCommand = new SqlCommand();`
 - Initialize a new instance of the SqlCommand class with the text of the query.
`SqlCommand myCommand = new SqlCommand(mySelectQuery);`
 - Initialize a new instance of the SqlCommand class with the text of the query and a SqlConnection.
`SqlCommand myCommand = new SqlCommand(mySelectQuery, yConnection);`
 - Initialize a new instance of the SqlCommand class with the text of the query, a SqlConnection, and the SqlTransaction.
`SqlCommand myCommand = new SqlCommand(mySelectQuery, yConnection, myTrans);`

SqlCommand Object (cont.)

- SqlCommand features the following methods for executing commands on a SQL Server database:

Item	Description
ExecuteReader	Executes commands that return rows.
ExecuteNonQuery	Executes commands such as Transact-SQL INSERT, DELETE, UPDATE statements.
ExecuteScalar	Retrieves a single value (for example, an aggregate value) from a database.
ExecuteXmlReader	Sends the CommandText to the Connection and builds an XmlReader object.

SqlCommand Object Adding Parameters

- Using parameterized queries is a three step process:

1. Construct the SqlCommand command string with parameters:

- Example:

```
SqlCommand cmd = new SqlCommand( "select * from Customers where city =  
@City", conn);
```

2. Declare a SqlParameter object, assigning values as appropriate:

- Example:

```
SqlParameter param = new SqlParameter();  
param.ParameterName = "@City";  
param.Value = inputCity;
```

3. Assign the SqlParameter object to the SqlCommand object's Parameters property:

- Example:

```
cmd.Parameters.Add(param);
```

Reading Data with SqlDataReader

- A SqlDataReader is a type that is good for reading data in the most efficient manner possible.
- Data can be read only in a forward stream of rows. This makes SqlDataReader very fast and efficient
- To create a SqlDataReader, you must call the ExecuteReader method of the SqlCommand object, **rather than directly using a constructor**:
 - For example,:
`SqlDataReader myReader = myCommand.ExecuteReader();`
creates a SqlDataReader myReader

Reading Data with SqlDataReader (cont.)

- As the SqlDataReader returns data via a sequential stream, the data must be read row-by-row:
 - Once a row has been read, the previous row is no longer available.
 - To read that row again, you would have to create a new instance of the SqlDataReader and read through the data stream again.
 - Example of using SqlDataReader to read data:

```
while (myReader.Read())  
{  
    Console.WriteLine(myReader.GetInt32(0) + ", " + myReader.GetString(1));  
}
```
- While the SqlDataReader is in use, the associated SqlConnection is busy serving the SqlDataReader:
 - No other operations can be performed on the SqlConnection other than closing it.
 - This is the case until the Close method of the SqlDataReader is called.

Implementation of Connected Data Management

```
using System;  
using System.Data;  
using System.Data.SqlClient;
```

```
namespace TestDBApps  
{
```

```
    class TestDBClass  
    {
```

```
        static void Main()  
        {
```

```
            // conn and reader declared outside try  
            // block for visibility in finally block  
            SqlConnection custConn = null;  
            SqlDataReader custReader = null;
```

```
            string inputCity = "London";  
            try
```

```
            {
```

Creating a new command object

// Instantiate and open connection

```
            custConn = new SqlConnection("Server=(local);DataBase=Northwind;Integrated Security=SSPI");  
            custConn.Open();
```

// Declare command object with parameter

```
            SqlCommand custCmd = new SqlCommand("select * from Customers where city = @City", custConn);
```

Setting the input filter criteria

// Define parameters used in command object

```
            SqlParameter custParam = new SqlParameter();  
            custParam.ParameterName = "@City";  
            custParam.Value = inputCity;
```

Creating a new connection

Implementation of Connected Data Management (cont.)

// Add new parameter to command object

```
custCmd.Parameters.Add(custParam);
```

// Get data stream

```
custReader = custCmd.ExecuteReader();
```

// Access each record

```
while (custReader.Read())
```

```
{
```

```
    Console.WriteLine("{0}, {1}", custReader["CompanyName"],  
                      custReader["ContactName"]);
```

```
}
```

```
}
```

```
finally
```

```
{
```

// Close reader

```
if (custReader != null)
```

```
{
```

```
    custReader.Close();
```

```
}
```

// Close connection

```
if (custConn != null)
```

```
{
```

```
    custConn.Close();
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

Adding the parameter to the command object

Looping through the reader

Implementation of Connected Data Management by “using block”

```
using System;  
using System.Data;  
using System.Data.SqlClient;
```

```
namespace TestDBApps  
{
```

```
    class TestDBClass  
    {
```

```
        static void Main()  
        {
```

```
            // conn and reader declared outside try  
            SqlConnection custConn = null;  
            SqlDataReader custReader = null;
```

```
            string inputCity = "London";
```

```
            // Instantiate and open connection
```

```
            using (custConn = new SqlConnection("Server=(local);DataBase=Northwind;Integrated Security=SSPI"))
```

```
            {  
                custConn.Open();
```

```
                // Declare command object with parameter
```

```
                SqlCommand custCmd = new SqlCommand("select * from Customers where city = @City", custConn);
```

```
                // Define parameters used in command object
```

```
                SqlParameter custParam = new SqlParameter();
```

```
                custParam.ParameterName = "@City";
```

```
                custParam.Value = inputCity;
```

A connection should be opened inside a using block to ensure that the connection is closed when the code exits the block which includes exceptions too.

Implementation of Connected Data Management by “using block” (cont.)

```
// Add new parameter to command object
custCmd.Parameters.Add(custParam);

// Get data stream
custReader = custCmd.ExecuteReader();

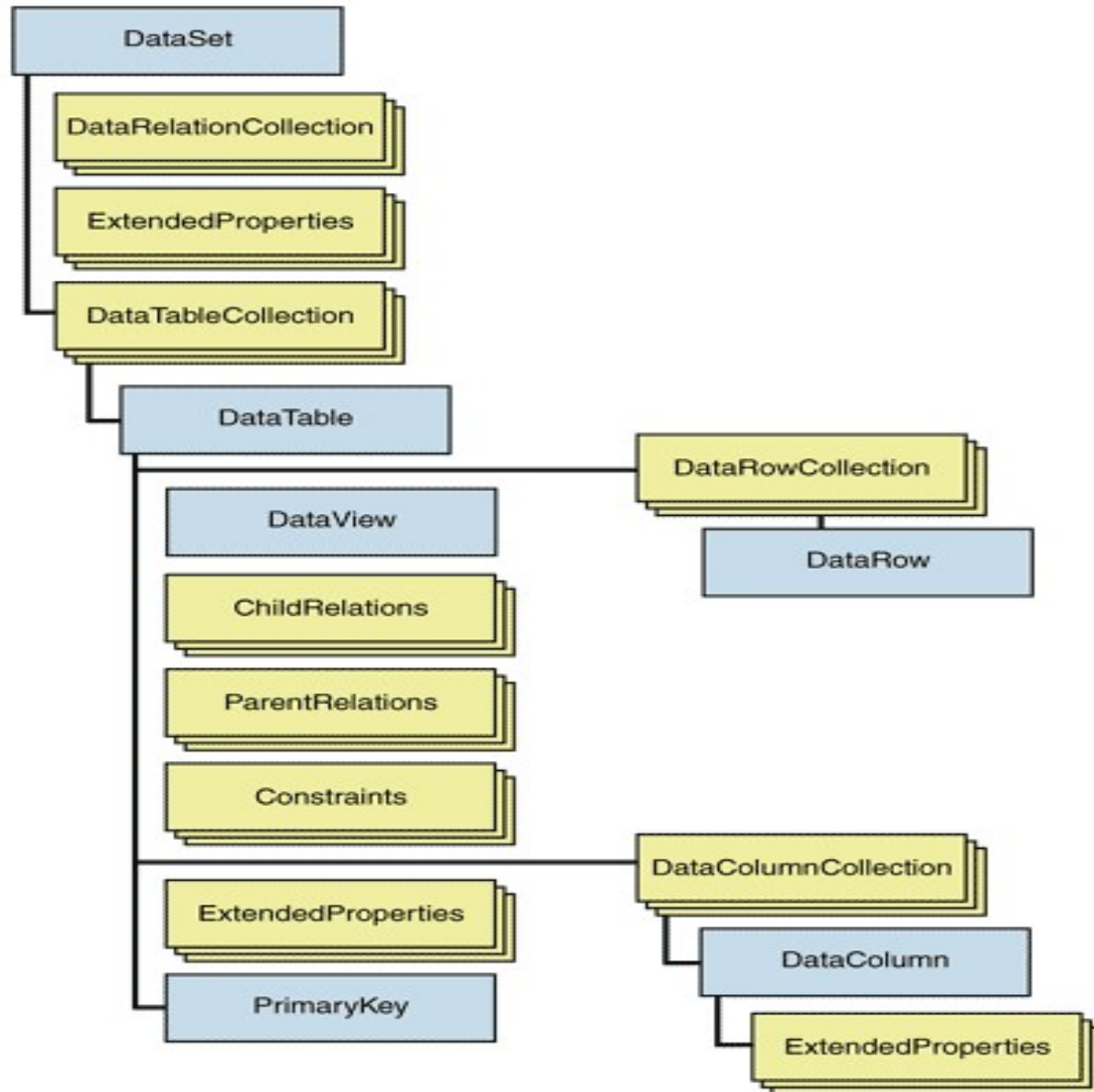
// Access each record
while (custReader.Read())
{
    Console.WriteLine("{0}, {1}", custReader["CompanyName"],
                      custReader["ContactName"]);
}
```

```
}
}
}
```

Working with Disconnected Data

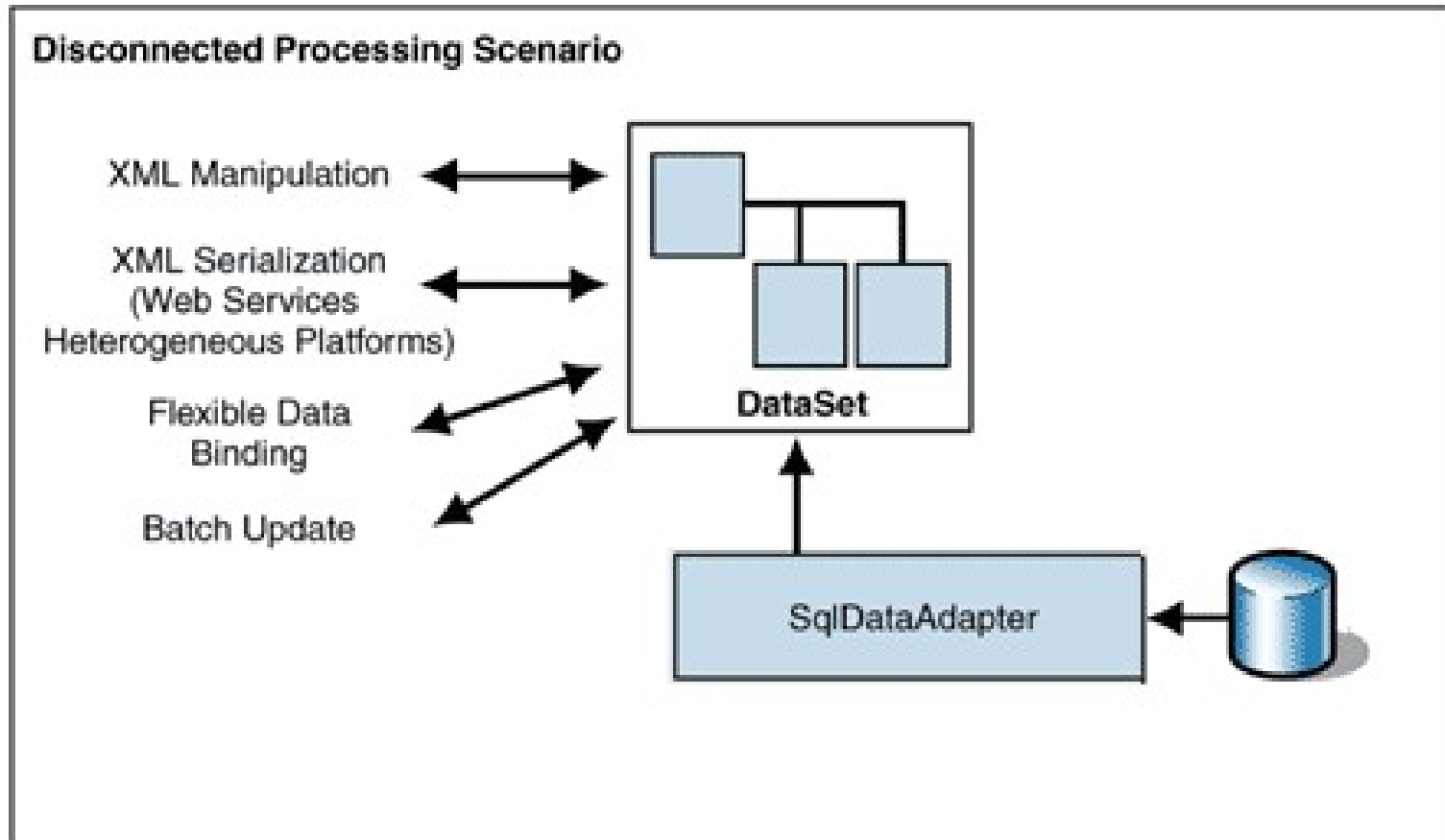
- In a disconnected architecture the connection to the database is not open during the time the data is used by the application:
 - Once the data is read from the database, the **data is held in memory** and the connection is closed. Then the application can update the data in memory.
 - If changes are to be updated to the database, a connection is opened again and the update occurs.
- Disconnected data management is handled effectively by the use of DataSet and SqlDataAdapter:
 - A DataSet is an in-memory data store that can hold numerous tables.
 - DataSets only hold data and do not interact with a data source.
 - The SqlDataAdapter manages connections with the data source and gives us disconnected behavior.
 - The SqlDataAdapter opens a connection only when required and closes it as soon as it has performed its task.

Working with Disconnected Data - DataSet Object Model



Working with Disconnected Data

Disconnected Processing Scenario



- In the this illustration, the SqlDataAdapter fills the DataSet from the database. The DataSet is then used to perform various operations (e.g., serialization, databinding, XML manipulation).

Working with Disconnected Data

- The following steps explain disconnected data management when data is read from the database. The following changes are made and then updated in the database:

1. Create a DataSet object:

```
DataSet custDataSet = new DataSet();
```

2. Create A SqlDataAdapter - The SqlDataAdapter holds the SQL commands and connection object for reading and writing data:

```
SqlDataAdapter custDA = new SqlDataAdapter("select CustomerID,  
Company Name from Customers", conn);
```

3. Construct the SqlCommandBuilder.:

```
SqlCommandBuilder cmdBldr = new SqlCommandBuilder(custDA);
```

Working with Disconnected Data (cont.)

4. Fill the DataSet – Use the Fill method of the SqlDataAdapter to fill the dataset:

```
custDA.Fill(custDataSet, "Customers");
```

5. Use the DataSet – in the application and make modifications to the data in the dataset.

6. Update the Changes back to the database – Use the Update method of the SqlDataAdapter to update the changes back to the database:

```
custDA.Update(custDataSet, "Customers");
```

Example for Disconnected Data

```
namespace ConnectedDatabaseProject
{
    class DisconnectedDataDemo
    {
        static void Main()
        {
            //Declare the data objects
            SqlConnection connection = null;
            SqlDataAdapter dataAdapter = null;
            DataSet dataset = null;
            try
            {
                //initilaize the connection
                connection = new SqlConnection(@"data source
= SQLSVR2005\SQLEXPRESS;initial catalog = pubs;
integrated security = SSPI");
                //open the connection
                connection.Open();
                Console.WriteLine("Connected");
                //initialize the dataadapter object
                dataAdapter = new SqlDataAdapter("select *
from authors", connection);
                //create the dataset
                dataset = new DataSet();
                //associate the dataset with the adapter
                dataAdapter.Fill(dataset, "authors");
            }
            // building a datatablereader to read the table
            DataTableReader datatablereader
            =dataset.CreateDataReader();
            if (datatablereader.HasRows)
            {
```

```
                //read the data in the datareader
                while (datatablereader.Read())
                {
                    Console.WriteLine(datatablereader["au_id"].ToString()
+ datatablereader["au_lname"].ToString() +
datatablereader["au_fname"].ToString());
                }
            }
            else
            {
                Console.WriteLine("No data Found");
            }
        }
        catch (Exception ex)
        { Console.WriteLine(ex.Message); }
        Finally
        {
            Console.WriteLine(ex.Message);
        }
    }
}
```

Introduction to DataTable

- The **SqlDataAdapter** stores the results of a query in a **DataTable**.
- The DataTable object is similar to the SqlDataAdapter object; either of them can be used for examining the results of a query.
- Both of these expose the results as a collection of rows and columns.
- The SqlDataAdapter's fill method creates a new DataTable.
- The DataTable object contains the following objects:
 - DataColumn
 - DataRow
 - Constraints

Introduction to DataColumn Object

- **DataColumn** objects define the schema for your **DataTable**.
- The SqlDataAdapter's Fill method creates a new DataTable and a DataColumn for each column in the results of the query.
- DataColumn object will have properties like name and datatype which is depicted in the code below.

- Example:

```
String strConn, strSQL ;  
strConn = " SQLSVR2005\SQLEXPRESS;initial catalog = pubs; integrated  
security = SSPI";  
strSQL = "SELECT au_id, au_lname, au_fname FROM authors";  
SQLDataAdapter da = new SqlDataAdapter(strSQL, strConn);  
DataSet ds = new DataSet();  
da.Fill(ds, "authors");  
foreach (DataColumn col In ds.Tables["authors"].Columns)  
    Console.WriteLine("{0} - {1}", col.ColumnName, col.DataType);
```

Introduction to DataRow Object

- The *DataRow* object allows viewing and modification of the content of the row in the DataTable.
- The DataTable's *Rows* Property returns a *DataRowCollection* object that contains a collection of *DataRows* objects.
- The following code reads data from the first row:

```
String strConn, strSQL ;  
strConn = " SQLSVR2005\SQLEXPRESS;initial catalog = pubs; integrated  
security = SSPI";  
strSQL = "SELECT au_id, au_lname, au_fname FROM authors";  
SQLDataAdapter da = new SqlDataAdapter(strSQL, strConn);  
DataSet ds = new DataSet();  
da.Fill(ds, "authors");  
DataRow row = ds.Tables["authors"].Row[0];  
Console.WriteLine("Author Id : {0} " , row["au_id"]);  
Console.WriteLine("Last Name : {0} " , row["au_lname"]);
```

Using Stored Procedures

- A stored procedure is a pre-defined, reusable routine that is stored in a database. SQL Server compiles stored procedures, which makes them more efficient to use.
- To use Stored Procedures for data access and to update the CommandType property of the SqlCommand, the object must be set to StoredProcedure.
- Parameters are passed to stored procedures using the SqlParameter object.

Implementation using Stored Procedure

```
using System;
using System.Data;
using System.Data.SqlClient;
```

```
namespace TestStoredProcApps
{
```

```
    class TestStoredProcedure
    {
```

```
        static void Main()
        {
```

```
            SqlConnection custConn = null;
            SqlDataReader custReader = null;
```

```
            // typically obtained from user
```

```
            // input, but we take a short cut
```

```
            string custId = "FURIB";
```

```
            Console.WriteLine("\nCustomer Order History:\n");
```

```
            try
```

```
            {
```

```
                // Create and open a connection object
```

```
                custConn = new SqlConnection("Server=(local);DataBase=Northwind;Integrated Security=SSPI");
                custConn.Open();
```

```
                // Create a command object identifying
```

```
                // the stored procedure
```

```
                SqlCommand custCmd = new SqlCommand("CustOrderHist", custConn);
```

```
                // Set the command type to stored procedure
```

```
                custCmd.CommandType = CommandType.StoredProcedure;
```

```
                // Add parameter to command, which will be passed to the stored procedure
```

```
                custCmd.Parameters.Add(new SqlParameter("@CustomerID", custId));
```

Create a new connection

Creating a command object with the stored procedure name as one of the parameters

Setting the command type as StoredProcedure

Implementation using Stored Procedure

```
        // Execute the command
        custReader = custCmd.ExecuteReader();

        // Iterate through results, printing each to console
        while (custReader.Read())
        {
            Console.WriteLine(
                "Product: {0,-35} Total: {1,2}",
                custReader["ProductName"],
                custReader["Total"]);
        }
    }
    finally
    {
        // Close the reader
        if (custReader != null)
        {
            custReader.Close();
        }

        // Close the connection
        if (custConn != null)
        {
            custConn.Close();
        }
    }
}
}
```

Implementation using Stored Procedure

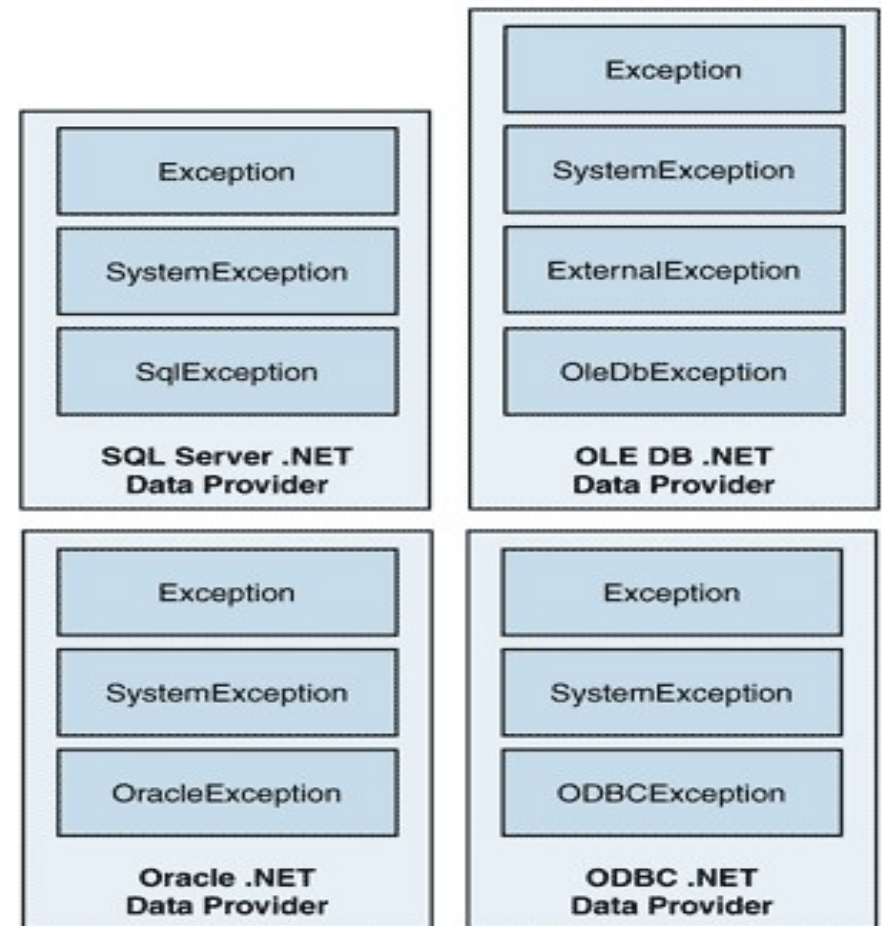
- Output:

Customer Order History:

Product: Camembert Pierrot	Total: 22
Product: Chef Anton's Cajun Seasoning	Total: 16
Product: Chocolate	Total: 6
Product: Gnocchi di nonna Alice	Total: 42
Product: Guaraná Fantástica	Total: 10
Product: Gula Malacca	Total: 55
Product: Ikura	Total: 24
Product: Ipoh Coffee	Total: 3
Product: Lakkalikööri	Total: 10
Product: Louisiana Fiery Hot Pepper Sauce	Total: 55
Product: Original Frankfurter grüne Soße	Total: 7
Product: Pavlova	Total: 40
Product: Raclette Courdavault	Total: 9
Product: Scottish Longbreads	Total: 10
Product: Steeleye Stout	Total: 20
Product: Tourtière	Total: 20

Exception Handling in ADO.NET

- The .NET data providers throw provider-specific exception types:
 - For example, the SQL Server .NET Data Provider throws `SqlException` objects whenever the SQL Server returns an error condition.
 - Similarly, the OLE DB .NET Data Provider throws exceptions of type `OleDbException`, which contain details exposed by the underlying OLE DB provider.
 - The .NET data provider exception hierarchy is shown in the illustration.

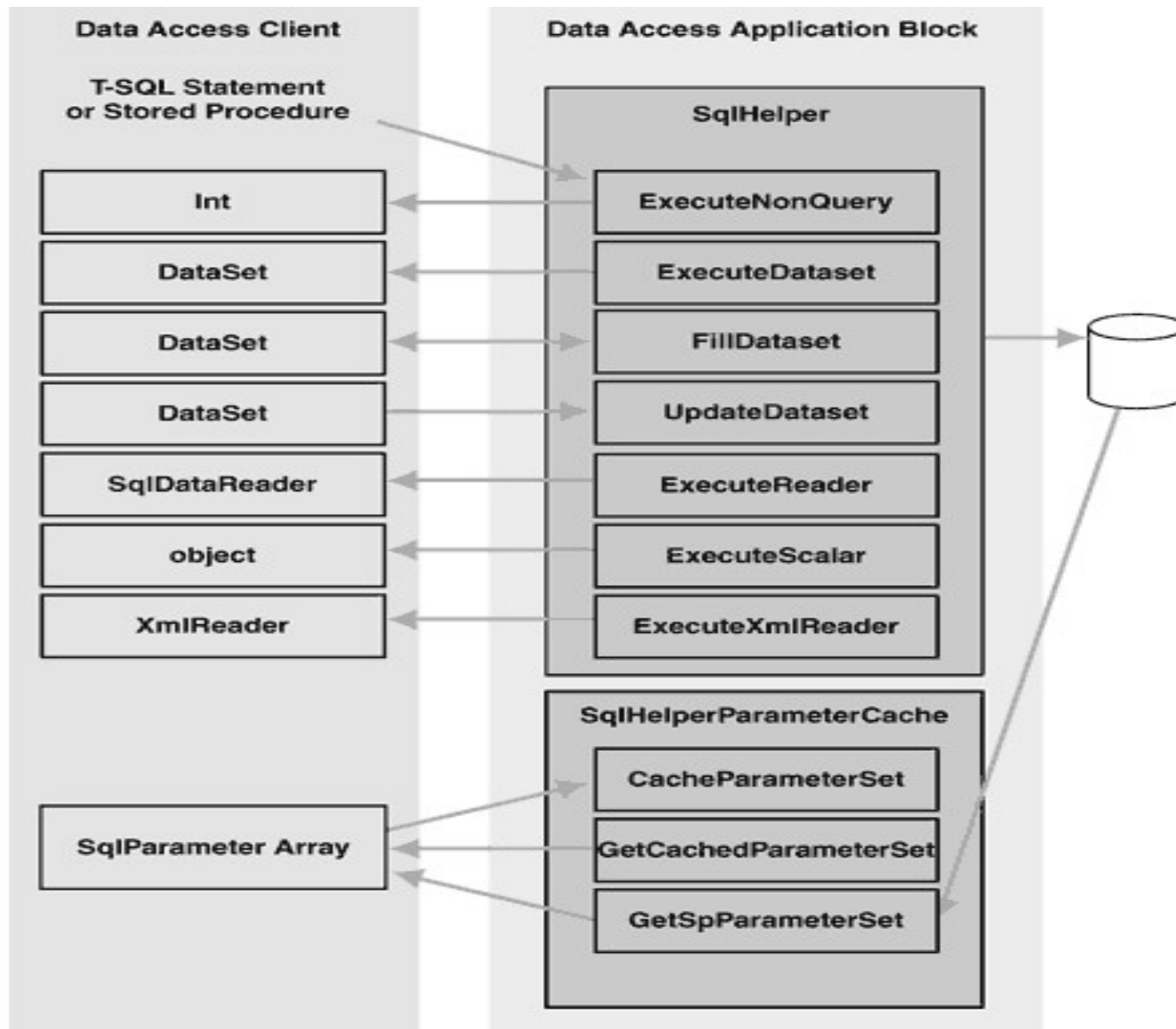


Data Access Application Block

- The Data Access Application Block is a managed component that exposes classes with methods:
 - The classes help you execute commands against a Microsoft SQL Server database in a robust and efficient manner.
 - It incorporates Microsoft ADO.NET data access best practices and uses data access patterns.
- The Data Access Application Block consists of two classes:
 - SqlHelper class, which provides methods for executing database commands.
 - SqlHelperParameterCache class, which provides parameter management functionality.
- Note: The Data Access Application Block is not a part of .NET Framework.

Data Access Application Block

Main Elements



Data Access Application Block

Methods in SqlHelper (1 of 3)

- The methods available in SqlHelper are:
 - ExecuteNonQuery - This method is used to execute commands that do not return any rows or values. They are generally used to perform database updates, but they can also be used to return output parameters from stored procedures.
 - ExecuteReader - This method is used to return a SqlDataReader object that contains the resultset returned by a command.
 - ExecuteDataset - This method returns a DataSet object that contains the resultset returned by a command.
 - ExecuteScalar - This method returns a single value. The value is always the first column of the first row returned by the command.
 - ExecuteXmlReader - This method returns an XML fragment from a FOR XML query.

Data Access Application Block

Methods in SqlHelper (2 of 3)

- FillDataset - This method is similar to ExecuteDataset, except that instead of returning a new DataSet object, it adds data to an existing DataSet.
- UpdateDataset - This method propagates changes recorded in a DataSet back to database.
- CreateCommand - This method allows you to build the command arguments required by UpdateDataset.
- ExecuteNonQueryTypedParams - This method is similar to ExecuteNonQuery method, except that it takes its parameters from a DataRow argument.
- ExecuteDatasetTypedParams - This method is similar to ExecuteDataset method, except that it takes its parameters from a DataRow argument.

Data Access Application Block

Methods in SqlHelper (3 of 3)

- `ExecuteReaderTypedParams` - This method is similar to `ExecuteReader` method, except that it takes its parameters from a `DataRow` argument.
- `ExecuteScalarTypedParams` - This method is similar to `ExecuteScalar` method, except that it takes its parameters from a `DataRow` argument.
- `ExecuteXmlReaderTypedParams` - This method is similar to `ExecuteXmlReader` method, except that it takes its parameters from a `DataRow` argument.

Data Access Application Block Referencing in C#.NET Project

- The following steps must be performed when creating a data client using a Data Access Application Block:
 1. Set a reference to the Data Access Application Block assembly `Microsoft.ApplicationBlocks.Data.dll`.
 2. Add a using statement at the top of your data access client source file(s) to reference the `Microsoft.ApplicationBlocks.Data` namespace. All Data Access Application Block types are located within this namespace.
 3. Add a using statement to reference the `System.Data` and `System.Data.SqlClient` namespaces. The `SqlHelper` methods use various types from these namespaces as input and return parameters.
 4. As an option, you may add a using statement to reference the `System.Xml` namespace. This statement is required if you use the `SqlHelper ExecuteXmlReader` methods.

Transactions in ADO.NET

- ADO.NET supports Transactions which are defined in the .NET Framework.
- Transactions map to local or database transactions which affect operations targeting a single database.
- Transactions are supported by Transaction classes and methods provided by ADO.NET.
- ADO.NET supports Transaction class and supports BeginTransaction(), commit() and Rollback() methods.

Transactions Implementation

```
//Initialize SqlCommand,SQLConnection objects
    SqlTransaction trans = null;
    try
    {
        Connection.Open();
        trans = Connection.BeginTransaction();
        SqlHelper.ExecuteNonQuery(trans, CommandType.StoredProcedure, "usp_LoanDelete", parLoanId);
        trans.Commit();
        Connection.Close();
        return true;
    }
    catch (Exception ex)
    {
        trans.Rollback();
        throw ex;
    }
finally
{
    if (Connection.State == ConnectionState.Open)
    {
        Connection.Close();
    }
}
```

Key Points

- ADO.NET is an object-oriented set of libraries that allows you to interact with data sources.
- ADO.NET has several data providers optimized for different databases.
- SqlDataReader can be used for forward-only mechanism of reading data. The connection remains open as long as the SqlDataReader is active.
- Disconnected data access is achieved by SqlDataAdapter and DataSets.
- By using the Data Access Application Block in your applications, you can:
 - Minimize the data access code you need to write, often to a single line.
 - Ensure that your data access logic is implemented in an efficient and effective manner.